

Hello



PRACTICAL LOGBOOK

Fundamentals of JavaScript

Scripting and Front-End Basics

Programme Details

Skills Programme: JavaScript Programmer
Programme Code: SP-220373
NQF Level: 4
Credits: 60
Practical Modules Covered: PM-01 to PM-04

Learner full name	Shane Davis
ID / passport number	8309175131082
Employer / department	Redpanda Type text here
Cohort / intake	Group 1
Facilitator	Mr Mahomed H. G. Nabee
Assessor	Mr Mahomed H. G. Nabee
Start date	20 July 2026
Completion date	20 July 2026

These concepts introduce dynamic programming, reusable program functions, and rigorous code testing to build reliable retail software.

Purpose and Use of this Logbook

This logbook records the learner's completion of the practical modules in the JavaScript Programmer Skills Programme. It provides a structured record of practical activities, evidence produced, learner reflections, facilitator observations, assessment decisions, corrective actions and final sign-off.

Important Quality Note

This logbook is designed to align with QCTO occupational learning principles for practical learning, internal assessment, evidence collection and competency recording. It must be used together with the registered curriculum, the provider's assessment instruments, moderation requirements and applicable QCTO/SETA instructions. It does not replace any official curriculum or assessment specification.

Roles and Responsibilities

Role	Responsibility
Learner	Complete activities honestly, retain authentic evidence, reflect on learning, respond to feedback and obtain sign-off.
Facilitator / Mentor	Guide practice, observe performance, provide developmental feedback and verify participation.
Assessor	Judge evidence against the stated criteria, record competent or not-yet-competent decisions and specify corrective action.
Moderator / Quality Reviewer	Check that assessment decisions are fair, valid, reliable, consistent and supported by evidence.
Employer / Workplace Representative	Where applicable, confirm workplace relevance, attendance and authenticity of activities performed.

Learner Declaration

I declare that the work recorded in this logbook is my own, except where collaboration or external assistance is clearly acknowledged. I understand that copied, fabricated or misleading evidence may be rejected and may be managed according to the provider's assessment and disciplinary procedures.

Learner name	Shane Davis
Learner signature	
Date	23 July 2026
Facilitator witness	Ryan Williams
Witness signature	

Assessment Appeals and Reassessment

A learner who is assessed as Not Yet Competent should receive clear feedback and a reasonable opportunity to correct gaps or complete reassessment according to the provider's assessment policy. Assessment appeals must follow the provider's formal appeals process.

Evidence Rules

- ☐ Evidence must be valid: it must relate directly to the practical task and criteria.
- ☐ Evidence must be authentic: it must be the learner's own work.
- ☐ Evidence must be current: it must demonstrate present competence.
- ☐ Evidence must be sufficient: enough evidence must be available to support the decision.
- ☐ Files should be named clearly and retained in the learner's portfolio or LMS submission area.
- ☐ Screenshots should show the relevant output, date/context where possible, and not expose confidential customer data.
- ☐ GitHub evidence should include meaningful commits and, where required, branch or pull-request evidence.

Programme Practical Module Map

Module	Title	Credits	Main practical output
PM-01	JavaScript Programming Fundamentals	6	Set up tools; create first HTML/CSS/JS page; use browser tools and GitHub.
PM-02	Programme with JavaScript	6	Apply variables, data types, operators, conditions, functions, arrays and loops.
PM-03	Intermediate Programming with JavaScript	12	Apply debugging, validation, DOM manipulation, events and basic API communication.
PM-04	Projects with JavaScript	12	Plan, build, test, document and present one integrated browser-based project.

Overall Practical Evidence Register

Evidence No.	Module	Description	File / link	Date	Verified by

Evidence No.	Module	Description	File / link	Date	Verified by

PM-01: JavaScript Programming Fundamentals

Module Purpose

To set up and use the basic tools required for browser-based JavaScript development, create a first interactive page and demonstrate basic Git/GitHub version control.

Activity 1.1: Development Environment and Project Structure

Purpose of the Activity

Create and verify a working beginner JavaScript development environment.

Instructions to the learner

- ☐ Create a clearly named project folder.
- ☐ Create index.html, style.css and script.js.
- ☐ Open the folder in VS Code.
- ☐ Connect the CSS and JavaScript files correctly.
- ☐ Run the page in Chrome or Edge.
- ☐ Open browser Developer Tools and identify the Console and Elements tabs.

Required evidence

- ☐ Screenshot of the project folder and VS Code file structure.
- ☐ Screenshot of the page running in the browser.
- ☐ Screenshot of Developer Tools with a console message.
- ☐ The three source files.

Competency criteria

- ☐ Correct folder and file structure.
- ☐ Files are linked correctly.
- ☐ Page loads without blocking errors.
- ☐ Learner can explain the role of HTML, CSS and JavaScript.

Learner Activity Record

Date started	20 July 2026
Date completed	20 July 2026
Location / platform	Redpanda Software
Time spent (hours)	1 Hour
GitHub repository / link	https://github.com/welkinspirit-hash/My-First-JS-Project
Evidence file names	Activity 1.1 Development Environment and Project Structure

Challenges experienced	Being exposed to this for the first time.
How challenges were resolved	Make notes and ask for help when needed .

Learner Reflection

What did you complete in this activity?

Create an html/css and js and setup a github repo.

Which JavaScript concepts did you apply?

Variables,data types,functions,buttons

What worked well?

Learning new concepts.

What error or difficulty did you encounter?

Since this my first time being exposed to this, grasping certain concepts.

How would this skill be useful in a retail or workplace environment?

Using it on the retail in store screen to enhance the software experience.

Facilitator / Assessor Verification

Criterion	Competent	Not Yet Competent	Comments / corrective action
Correct folder and file structure.	☐	☐	
Files are linked correctly.	☐	☐	
Page loads without blocking errors.	☐	☐	
Learner can explain the role of HTML, CSS and JavaScript.	☐	☐	

Overall result: Competent / Not Yet Competent	
Assessor / facilitator name	
Signature	

Date	
Learner acknowledgement and signature	

Activity 1.2: My First Interactive Web Page

Purpose of the Activity

Build a simple page introducing the learner or department and add basic interaction.

Instructions to the learner

- ☐ Add a heading and short paragraph.
- ☐ Apply basic CSS styling.
- ☐ Add a button.
- ☐ Use JavaScript to show an alert when the button is clicked.
- ☐ Write a meaningful console.log() message.
- ☐ Use comments and neat indentation.

Required evidence

- ☐ HTML, CSS and JavaScript files.
- ☐ Screenshot before and after interaction.
- ☐ Screenshot of console output.
- ☐ Short explanation of the code.

Competency criteria

- ☐ Page contains required HTML content.
- ☐ CSS improves readability.
- ☐ Button interaction works.
- ☐ Code is readable and commented.

Learner Activity Record

Date started	22 July 2026
Date completed	22 July 2026
Location / platform	Redpanda Software
Time spent (hours)	1 Hour
GitHub repository / link	https://github.com/welkinspirit-hash/My-First-JS-Project/
Evidence file names	Activity 1.2 My First Interactive Web Page
Challenges experienced	Being exposed to more challenging experiences.
How challenges were resolved	Utilising the workbook and receiving assistance from peers.

Learner Reflection

What did you complete in this activity?

Understanding how web browser works

Which JavaScript concepts did you apply?

Html,CSS and JS and how certain variables work.

What worked well?

Changing the theme of the webpage.

What error or difficulty did you encounter?

Broken script paths: The browser fails to display the alert window because the HTML file cannot locate the separate JavaScript file due to a typo in the file path

How would this skill be useful in a retail or workplace environment?

It will enhance the user experience Type text here

Facilitator / Assessor Verification

Criterion	Competent	Not Yet Competent	Comments / corrective action
Page contains required HTML content.	☐	☐	
CSS improves readability.	☐	☐	
Button interaction works.	☐	☐	
Code is readable and commented.	☐	☐	

Overall result: Competent / Not Yet Competent	
Assessor / facilitator name	
Signature	
Date	
Learner acknowledgement and signature	

Activity 1.3: Git and GitHub Basics

Purpose of the Activity

Create a repository and record the development history.

Instructions to the learner

- ☐ Create a GitHub repository.
- ☐ Add the project files.
- ☐ Create at least three meaningful commits.
- ☐ Create a branch and make one change.
- ☐ Push the branch and open a basic pull request, where required.
- ☐ Use clear commit messages.

Required evidence

- ☐ Repository link.
- ☐ Screenshot of commit history.
- ☐ Screenshot of branch or pull request.
- ☐ README or project description.

Competency criteria

- ☐ Repository contains correct files.
- ☐ Commit history shows progressive work.
- ☐ Branching evidence is present.
- ☐ Learner can explain Git versus GitHub.

Learner Activity Record

Date started	22 July 2026
Date completed	22 July 2026
Location / platform	Redpanda Software
Time spent (hours)	2 Hours
GitHub repository / link	https://github.com/welkinspirit-hash/My-First-JS-Project/
Evidence file names	Activity 1.3 Git and GitHub Basics
Challenges experienced	To understand different challenges being experienced on Git Hub.
How challenges were resolved	Utilising the workbook and receiving assistance from peers.

Learner Reflection

What did you complete in this activity?

How to create a git hub repository,add project files and commit changes utilising git pull/push/commit.

Which JavaScript concepts did you apply?

Version history tracking: Creating a sequence of descriptive commits builds an unchangeable timeline to monitor code changes over time.

What worked well?

Safe code reviews: Opening a clear pull request gives a structured framework to verify and check experimental branch updates before merging them into production

What error or difficulty did you encounter?

Authentication errors: Pushing a local branch to GitHub gets blocked due to missing or expired personal access tokens in the terminal.

How would this skill be useful in a retail or workplace environment?

Using Git version control prevents team coordination chaos and safeguards digital business assets during system modifications.

Facilitator / Assessor Verification

Criterion	Competent	Not Yet Competent	Comments / corrective action
Repository contains correct files.	☐	☐	
Commit history shows progressive work.	☐	☐	
Branching evidence is present.	☐	☐	
Learner can explain Git versus GitHub.	☐	☐	

Overall result: Competent / Not Yet Competent	
Assessor / facilitator name	
Signature	
Date	
Learner acknowledgement and signature	

Type text here

PM-02: Programme with JavaScript

Module Purpose

To apply JavaScript syntax and programming fundamentals in small exercises and a Support Ticket Priority Checker.

Activity 2.1: Variables, Data Types and Operators

Purpose of the Activity

Capture and calculate workplace or retail information using variables and operators.

Instructions to the learner

- ☐ Declare values using let and const.
- ☐ Use strings, numbers and Booleans.
- ☐ Perform arithmetic calculations such as totals, VAT or quantities.
- ☐ Use comparison and logical operators.
- ☐ Output results to the console or page.
- ☐ Add comments explaining the code.

Required evidence

- ☐ JavaScript exercise file.
- ☐ Screenshot of at least three outputs.
- ☐ Short explanation identifying each data type and operator used.

Competency criteria

- ☐ Variables are named meaningfully.
- ☐ Data types match the values stored.
- ☐ Calculations are correct.
- ☐ Outputs are clear and code is readable.

Learner Activity Record

Date started	31st August 2026
Date completed	02 September 2026
Location / platform	Redpanda Software
Time spent (hours)	2-3 hours
GitHub repository / link	
Evidence file names	PM-02: Programme with JavaScript
Challenges experienced	Scope Confusion: Distinguishing exactly when to use let versus const for variables that might change later, such as running totals

How challenges were resolved

Strict Variable Declaration: Enforced the rule of using const by default for fixed values like VAT rates, and let only for values that explicitly need reassignment.

Learner Reflection

What did you complete in this activity?

I applied data storage declaration rules, diverse value types, mathematical checkout calculations, and conditional evaluations.

Which JavaScript concepts did you apply?

Applying block-scoped variable declarations, primitive data types, arithmetic calculation rules, and relational tracking logic

What worked well?

Structuring data and using foundational math operations allowed for accurate and transparent calculations of complex workplace transactions.

What error or difficulty did you encounter?

Applying certain logic

How would this skill be useful in a retail or workplace environment?

Learning these programming fundamentals directly enables you to build and maintain modern retail systems like Point of Sale (POS) software, inventory trackers, and e-commerce platforms.

Facilitator / Assessor Verification

Criterion	Competent	Not Yet Competent	Comments / corrective action
Variables are named meaningfully.	☐	☐	
Data types match the values stored.	☐	☐	
Calculations are correct.	☐	☐	
Outputs are clear and code is readable.	☐	☐	

Overall result: Competent / Not Yet Competent	
Assessor / facilitator name	
Signature	
Date	

**Learner acknowledgement
and signature**

Activity 2.2: Conditions and Functions

Purpose of the Activity

Use reusable logic to make decisions.

Instructions to the learner

- ☐ Write if, else if and else logic.
- ☐ Use a switch statement in one exercise.
- ☐ Create a function with parameters.
- ☐ Return a value from a function.
- ☐ Test normal and edge-case inputs.
- ☐ Document the purpose of each function.

Required evidence

- ☐ JavaScript source file.
- ☐ Screenshots showing different decision outcomes.
- ☐ Test table with expected and actual outcomes.

Competency criteria

- ☐ Conditions produce correct outcomes.
- ☐ Function is called correctly.
- ☐ Return value is used meaningfully.
- ☐ At least three test cases are recorded.

Learner Activity Record

Date started	31st August 2026
Date completed	02 September 2026
Location / platform	Redpanda Software
Time spent (hours)	2-3 Hours
GitHub repository / link	
Evidence file names	Activity 2.2: Conditions and Functions
Challenges experienced	Logic Traps and Overlaps: Managing complex if/else if/else conditions where the ordering of checks accidentally caused the wrong block of code to run first.
How challenges were resolved	Ordered Conditional Chains: Re-ordered conditional logic to evaluate the most specific boundaries first before falling back on general else conditions.

Learner Reflection

What did you complete in this activity?

These concepts introduce dynamic decision-making, reusable program functions, and rigorous code testing to build reliable retail software.

Which JavaScript concepts did you apply?

If/Else: Handles changing business rules dynamically.
 Switch: Matches exact inventory categories quickly.
 Functions: Bundles retail steps into reusable tools.

What worked well?

Task separation: Reusable functions keep checkout code clean.
 Faster processing: Switch statements match store items instantly.

What error or difficulty did you encounter?

Missing switch breaks: Forgetting break commands causes the program to accidentally roll over into wrong discount categories.

How would this skill be useful in a retail or workplace environment?

Automating business rules with functional programming logic streamlines day-to-day operations and eliminates manual calculation mistakes.

Facilitator / Assessor Verification

Criterion	Competent	Not Yet Competent	Comments / corrective action
Conditions produce correct outcomes.	☐	☐	
Function is called correctly.	☐	☐	
Return value is used meaningfully.	☐	☐	
At least three test cases are recorded.	☐	☐	

Overall result: Competent / Not Yet Competent	
Assessor / facilitator name	
Signature	
Date	
Learner acknowledgement and signature	

Activity 2.3: Arrays, Loops and Support Ticket Priority Checker

Purpose of the Activity

Process multiple tickets and assign appropriate priority messages.

Instructions to the learner

- ☐ Create an array of ticket priorities or ticket objects.
- ☐ Loop through all records.
- ☐ Use a function to determine the message for High, Medium and Low priority.
- ☐ Show the result in the console or page.
- ☐ Handle unexpected or missing priority values.
- ☐ Maintain a meaningful GitHub commit history.

Required evidence

- ☐ Completed Support Ticket Priority Checker files.
- ☐ Screenshots showing High, Medium, Low and invalid outcomes.
- ☐ GitHub repository and commit history.
- ☐ Short reflection on variables, functions, arrays, loops and conditions.

Competency criteria

- ☐ All required priority outcomes work.
- ☐ Array and loop process multiple items.
- ☐ Function and conditions are correctly structured.
- ☐ Unexpected input is handled clearly.

Learner Activity Record

Date started	31st August 2026
Date completed	27 September 2026
Location / platform	Redpanda Software
Time spent (hours)	2-3 Hours
GitHub repository / link	
Evidence file names	PM02 Programme with Javascript
Challenges experienced	Data Validation Vulnerabilities: Handling missing, misspelled, or null priority keys when looping through an array of ticket objects, which initially caused runtime errors.
How challenges were resolved	Defensive Fallback Logic: Implemented rigid default conditions (such as a catch-all "Invalid/Unknown Priority" message) to gracefully handle unexpected input data without breaking the program execution.

Learner Reflection

What did you complete in this activity?

You built an automated customer service ticketing program that categorizes issue logs and handles broken data entries seamlessly.

Which JavaScript concepts did you apply?

Applied data structures, looping mechanisms, execution functions, and input validation rules

What worked well?

Batch processing efficiency: A loop runs through hundreds of support tickets simultaneously without needing manual input for each entry

What error or difficulty did you encounter?

Undefined property crashes: The program breaks when a loop hits a ticket object entirely missing its priority property.

How would this skill be useful in a retail or workplace environment?

Automated inventory reordering: A loop checks the stock levels of all products, instantly flagging low-count items for urgent wholesale reordering.

Facilitator / Assessor Verification

Criterion	Competent	Not Yet Competent	Comments / corrective action
All required priority outcomes work.	☐	☐	
Array and loop process multiple items.	☐	☐	
Function and conditions are correctly structured.	☐	☐	
Unexpected input is handled clearly.	☐	☐	

Overall result: Competent / Not Yet Competent	
Assessor / facilitator name	
Signature	
Date	
Learner acknowledgement and signature	

PM-03: Intermediate Programming with JavaScript

Module Purpose

To apply debugging, validation, DOM manipulation, browser events, error handling and basic REST API communication.

Activity 3.1: Debugging and Error Handling

Purpose of the Activity

Find and correct JavaScript errors using systematic troubleshooting.

Instructions to the learner

- ☐ Open and read browser console errors.
- ☐ Use `console.log()`, `console.warn()` and `console.error()` appropriately.
- ☐ Use a breakpoint and step through code.
- ☐ Correct at least three seeded errors.
- ☐ Use `try...catch` for a controlled error example.
- ☐ Record the original error, cause and fix.

Required evidence

- ☐ Before-and-after screenshots.
- ☐ Debugging record for at least three errors.
- ☐ Corrected code.
- ☐ Screenshot of breakpoint or Sources panel.

Competency criteria

- ☐ Learner identifies relevant error details.
- ☐ Fix addresses the root cause.
- ☐ `try...catch` is used appropriately.
- ☐ Debugging record is clear and reproducible.

Learner Activity Record

Date started	15 September 2026
Date completed	27 September 2026
Location / platform	Redpanda Software
Time spent (hours)	2-3 hours
GitHub repository / link	
Evidence file names	PM-03 Projects with JavaScript

Challenges experienced	I had difficulty finding some JavaScript errors and understanding why the code was not working as expected.
How challenges were resolved	I used the browser console, console.log(), breakpoints and step-by-step debugging to find the cause and correct the errors.

Learner Reflection

What did you complete in this activity?

I found and fixed three seeded JavaScript errors and tested the corrected code.
I also used try...catch to handle an error.

Which JavaScript concepts did you apply?

I applied event handling, DOM manipulation, console debugging, breakpoints, try...catch, and error handling.

What worked well?

Using the browser console and console.log() helped me see what the code was doing and identify where the problem occurred

What error or difficulty did you encounter?

I encountered errors caused by incorrect or missing code, such as a missing event.preventDefault() and other seeded errors. I used debugging tools to identify and fix them.

How would this skill be useful in a retail or workplace environment?

Debugging helps identify and fix software problems quickly, reducing system downtime and helping business applications work correctly for users and customers.

Facilitator / Assessor Verification

Criterion	Competent	Not Yet Competent	Comments / corrective action
Learner identifies relevant error details.	☐	☐	
Fix addresses the root cause.	☐	☐	
try...catch is used appropriately.	☐	☐	
Debugging record is clear and reproducible.	☐	☐	

Overall result: Competent / Not Yet Competent	
Assessor / facilitator name	
Signature	

Date	
Learner acknowledgement and signature	

Activity 3.2: Department Contact Form with Validation

Purpose of the Activity

Build and validate an interactive workplace contact form.

Instructions to the learner

- ☐ Create fields for name, department, email, issue category and message.
- ☐ Prevent submission when required data is missing.
- ☐ Check that the email format is reasonable.
- ☐ Display specific error messages.
- ☐ Display a success message for valid input.
- ☐ Use DOM methods and a submit event listener.
- ☐ Test valid, invalid and boundary cases.

Required evidence

- ☐ HTML, CSS and JavaScript files.
- ☐ Screenshots of missing-field, invalid-email and successful submission states.
- ☐ Completed test table.
- ☐ GitHub commits.

Competency criteria

- ☐ All required fields are checked.
- ☐ Invalid email is rejected with helpful feedback.
- ☐ Valid input produces a success state.
- ☐ DOM updates and event handling work correctly.

Learner Activity Record

Date started	15 September 2026
Date completed	27 September 2026
Location / platform	Redpanda Software
Time spent (hours)	2-3 hours
GitHub repository / link	
Evidence file names	PM-03 Projects with JavaScript
Challenges experienced	Form Submission Control: Managing the default browser behavior, where unvalidated form submissions would instantly reload the page and erase user input.
How challenges were resolved	Event Prevention Management: Utilized JavaScript's event.preventDefault() within a submit event listener to pause default submission until all fields pass validation checks.

Learner Reflection

What did you complete in this activity?

I built an interactive workplace contact form using HTML, CSS, and JavaScript. The form collects user details (Name, Department, Email, Issue Category, Message) and validates the inputs dynamically before allowing submission.

Which JavaScript concepts did you apply?

- DOM Manipulation: Selecting elements using methods like `document.getElementById()` and dynamically altering text/styles to display error or success messages.
- Event Handling: Adding an `addEventListener` to listen for the form's submit event and using `event.preventDefault()` to stop automatic page reloads.
- Conditional Logic & Regular Expressions (Regex): Using `if/else` statements to check for missing inputs and applying Regex to check if the email format is valid.

What worked well?

The event listener effectively intercepted form submissions, allowing real-time feedback. Dynamically updating the DOM meant users received clear, immediate error messages next to invalid fields without needing to refresh the page.

What error or difficulty did you encounter?

- Page Refreshing: Initially, submitting the form caused the page to reload instantly, clearing all user inputs before validation could run.
- Regex Validation: Creating a pattern for email validation that correctly flagged invalid emails without breaking on valid ones.

How would this skill be useful in a retail or workplace environment?

Form validation ensures that customer inquiries, internal support tickets, or feedback submissions are captured accurately without missing key info (like an email address). This saves time, prevents database clutter with broken entries, and improves overall communication efficiency in a business setting.

Facilitator / Assessor Verification

Criterion	Competent	Not Yet Competent	Comments / corrective action
All required fields are checked.	☐	☐	
Invalid email is rejected with helpful feedback.	☐	☐	
Valid input produces a success state.	☐	☐	
DOM updates and event handling work correctly.	☐	☐	

Overall result: Competent / Not Yet Competent	
Assessor / facilitator name	
Signature	
Date	
Learner acknowledgement and signature	

Activity 3.3: DOM Events and Dynamic Content

Purpose of the Activity

Read and change page elements in response to user actions.

Instructions to the learner

- ☐ Select elements by ID or query selector.
- ☐ Change text or styles.
- ☐ Add and remove an item dynamically.
- ☐ Respond to click, input or change events.
- ☐ Use event.preventDefault() where appropriate.
- ☐ Keep the interface clear and usable.

Required evidence

- ☐ Source files.
- ☐ Screenshots showing dynamic changes.
- ☐ Short explanation of selectors and events used.

Competency criteria

- ☐ Correct elements are selected.
- ☐ Events trigger expected changes.
- ☐ Add/remove behaviour works.
- ☐ Code is organised and readable.

Learner Activity Record

Date started	15 September 2026
Date completed	27 September 2026
Location / platform	Redpanda Software
Time spent (hours)	2-3 hours
GitHub repository / link	
Evidence file names	PM-03 Projects with JavaScript
Challenges experienced	Targeting Correct Elements: Missing or mistyping ID and class naming conventions in querySelector, leading to null reference errors when attempting to select HTML elements.
How challenges were resolved	Precise Selector Strategies: Implemented rigorous checks utilizing explicit getElementById and structured querySelector syntax to guarantee accurate element targeting.

Learner Reflection

What did you complete in this activity?

I built an interactive web page that reads and updates content in real time based on user actions like clicks and text input. I added functionality to dynamically add and remove list items, update styles, and modify text content without reloading the page.

Which JavaScript concepts did you apply?

- DOM Selection: Used methods like `document.getElementById()` and `document.querySelector()` to find and manipulate elements.
- Event Listeners: Used `addEventListener()` to detect user interactions such as click, input, and change events.
- Dynamic DOM Manipulation: Used `createElement()`, `appendChild()`, and `.remove()` to add or delete elements on the fly.

What worked well?

Using `addEventListener()` allowed the page to feel fast and responsive. Updating text content and toggling styles directly through JavaScript provided immediate feedback whenever a user interacted with the page.

What error or difficulty did you encounter?

- Element Selection Errors: Getting null errors when attempting to select HTML elements using slightly incorrect ID or selector names.
- Event Behavior Issues: Buttons submitting data prematurely before `event.preventDefault()` was properly implemented

How would this skill be useful in a retail or workplace environment?

Dynamic DOM manipulation is essential for creating interactive workplace tools like digital shopping carts, task tracking lists, or inventory management dashboards. It allows staff or customers to add, update, or remove items in real time, keeping the interface clean and user-friendly without unnecessary page reloads.

Facilitator / Assessor Verification

Criterion	Competent	Not Yet Competent	Comments / corrective action
Correct elements are selected.	☐	☐	
Events trigger expected changes.	☐	☐	
Add/remove behaviour works.	☐	☐	
Code is organised and readable.	☐	☐	

Overall result: Competent / Not Yet Competent	
Assessor / facilitator name	
Signature	
Date	
Learner acknowledgement and signature	

Activity 3.4: Display Data from an API

Purpose of the Activity

Retrieve and display simple data from a public or mock API.

Instructions to the learner

- ☐ Use `fetch()` to send a GET request.
- ☐ Convert the response to JSON.
- ☐ Display selected fields on the page.
- ☐ Show a loading or status message.
- ☐ Handle a failed request with `catch` or `try...catch`.
- ☐ Do not expose confidential credentials or customer information.

Required evidence

- ☐ HTML, CSS and JavaScript files.
- ☐ Screenshot of API data displayed.
- ☐ Screenshot of error-handling output or simulated failure.
- ☐ Short explanation of request, response and JSON.

Competency criteria

- ☐ Request retrieves data successfully.
- ☐ JSON fields are displayed clearly.
- ☐ Failure is handled without crashing the page.
- ☐ Learner can explain the purpose of an API.

Learner Activity Record

Date started	15 September 2026
Date completed	27 September 2026
Location / platform	Redpanda Software
Time spent (hours)	2-3 hours
GitHub repository / link	
Evidence file names	PM-03 Projects with JavaScript
Challenges experienced	Asynchronous Execution Flaws: Forgetting that network requests take time, leading to attempts to read or display the API data before the <code>fetch()</code> promise had fully resolved.
How challenges were resolved	Structured Promise Chains: Implemented sequential <code>.then()</code> blocks and clean conversion methods to ensure incoming responses were fully converted to JSON before data processing.

Learner Reflection

What did you complete in this activity?

I retrieved data from a public/mock API using JavaScript and dynamically displayed selected JSON fields on a web page. I also added status/loading indicators and error-handling routines so the application handles successful and failed network requests gracefully.

Which JavaScript concepts did you apply?

- Fetch API (fetch()): Sent asynchronous GET requests to an external API endpoint to fetch raw data.
- Promises & JSON Parsing: Converted the raw API response stream into readable JSON objects using .json().
- Asynchronous Control (try...catch): Managed network outcomes and errors using try...catch or .catch() blocks to prevent app crashes.
- DOM Data Rendering: Injected specific JSON fields dynamically into targeted HTML elements.

What worked well?

Displaying a loading message immediately provided a responsive user experience while the request was in progress. Converting the API response straight into JSON made extracting and rendering specific fields straightforward.

What error or difficulty did you encounter?

- Unresolved Promises / Syntax Errors: Forgetting to await the .json() conversion step, which returned an empty Promise object instead of actual data.
- Unchecked HTTP Errors: Learning that fetch() doesn't automatically reject HTTP error statuses (like 404 or 500), requiring manual checks on response.ok.

How would this skill be useful in a retail or workplace environment?

Fetching data from APIs allows web applications to display real-time information—such as live product inventory, customer profile data, pricing updates, or order tracking information—directly from a central workplace database without needing manual page updates.

Facilitator / Assessor Verification

Criterion	Competent	Not Yet Competent	Comments / corrective action
Request retrieves data successfully.	☐	☐	
JSON fields are displayed clearly.	☐	☐	
Failure is handled without crashing the page.	☐	☐	
Learner can explain the purpose of an API.	☐	☐	

Overall result: Competent / Not Yet Competent	
Assessor / facilitator name	
Signature	
Date	
Learner acknowledgement and signature	

PM-04: Projects with JavaScript

Module Purpose

To consolidate the programme through one complete beginner-friendly workplace web application using HTML, CSS, JavaScript and GitHub.

Approved Project Selection

The learner must select and complete ONE of the following project options:

- ☐ Option 1: Helpdesk Ticket Logger
- ☐ Option 2: Department Resource Request Form
- ☐ Option 3: Basic Calculator or Leave Days Calculator
- ☐ Option 4: Mini Quiz Application

Selected option	Mini Quiz Application
Reason for selecting this option	I chose this project to build a practical tool that reinforces cybersecurity awareness while applying JavaScript DOM manipulation and local storage persistence.
Facilitator approval	
Approval date	

Activity 4.1: Project Planning and Design

Purpose of the Activity

Define a clear, achievable project before development starts.

Instructions to the learner

- ☐ Write a short problem statement and target user description.
- ☐ List functional requirements.
- ☐ Draw a simple page sketch or wireframe.
- ☐ Identify variables, arrays or objects, functions, conditions, events and DOM changes.
- ☐ Create a test plan.
- ☐ Create the GitHub repository and README outline.

Required evidence

- ☐ Problem statement and requirements.
- ☐ Wireframe or page sketch.
- ☐ Planned data structure and logic.
- ☐ Test plan.
- ☐ Repository link.

Competency criteria

- ☐ Scope is appropriate for a beginner project.
- ☐ Requirements are clear and testable.
- ☐ Plan covers required JavaScript concepts.
- ☐ Evidence shows approval before full build.

Learner Activity Record

Date started	15 September 2026
Date completed	09 October 2026
Location / platform	Redpanda
Time spent (hours)	3-4 Hours
GitHub repository / link	
Evidence file names	PM-04 Projects with JavaScript
Challenges experienced	Managing missing or uninitialized values in localStorage when a new user visits the dashboard for the first time before taking any quiz
How challenges were resolved	Implemented safe fallback values and conditional logic checks (such as checking if (score > 0) and providing defaults) to handle null or uninitialized data gracefully without breaking DOM updates.

Learner Reflection

What did you complete in this activity?

I completed the project planning and design phase for the Mini Quiz Application, including defining the problem statement, functional requirements, text-based wireframe, JavaScript variable and logic mapping, test plan, and code structuring.

Which JavaScript concepts did you apply?

I applied DOM element selection (getElementById), dynamic text modification (textContent), localStorage data retrieval, conditional logic (if/else if/else), event listeners (click), and browser navigation control (window.location.href)

What worked well?

Integrating localStorage to seamlessly carry user login data and past scores across different pages worked smoothly, allowing for real-time personalization and conditional UI updates

What error or difficulty did you encounter?

Ensuring that elements parsed correctly when values were missing or uninitialized in localStorage initially caused minor rendering hitches, which I resolved by adding safe fallback checks

How would this skill be useful in a retail or workplace environment?

This skill is vital for building interactive employee training portals, tracking internal compliance or quiz progress, and creating dynamic dashboards that persist user session data locally

Facilitator / Assessor Verification

Criterion	Competent	Not Yet Competent	Comments / corrective action
Scope is appropriate for a beginner project.	☐	☐	

Criterion	Competent	Not Yet Competent	Comments / corrective action
Requirements are clear and testable.	☐	☐	
Plan covers required JavaScript concepts.	☐	☐	
Evidence shows approval before full build.	☐	☐	

Overall result: Competent / Not Yet Competent	
Assessor / facilitator name	
Signature	
Date	
Learner acknowledgement and signature	

Activity 4.2: Project Build and Version Control

Purpose of the Activity

Build the selected project progressively and record changes.

Instructions to the learner

- ☐ Create a working HTML structure and CSS layout.
- ☐ Implement input and validation.
- ☐ Use JavaScript functions and decision logic.
- ☐ Use an array or object where relevant.
- ☐ Use DOM manipulation and events.
- ☐ Create meaningful commits and at least one branch or pull request.
- ☐ Keep code readable and commented.

Required evidence

- ☐ Final source files.
- ☐ GitHub repository.
- ☐ Commit history.
- ☐ Branch or pull-request evidence.
- ☐ Screenshots of major features.

Competency criteria

- ☐ Core requirements are implemented.
- ☐ JavaScript concepts are applied correctly.
- ☐ Project remains within approved scope.
- ☐ Version-control history is authentic and progressive.

Learner Activity Record

Date started	
Date completed	
Location / platform	
Time spent (hours)	
GitHub repository / link	
Evidence file names	
Challenges experienced	
How challenges were resolved	

Learner Reflection

What did you complete in this activity?

Which JavaScript concepts did you apply?

What worked well?

What error or difficulty did you encounter?

How would this skill be useful in a retail or workplace environment?

Facilitator / Assessor Verification

Criterion	Competent	Not Yet Competent	Comments / corrective action
Core requirements are implemented.	☐	☐	
JavaScript concepts are applied correctly.	☐	☐	
Project remains within approved scope.	☐	☐	
Version-control history is authentic and progressive.	☐	☐	

Overall result: Competent / Not Yet Competent	
Assessor / facilitator name	
Signature	
Date	
Learner acknowledgement and signature	

Activity 4.3: Project Testing, Debugging and Improvement

Purpose of the Activity

Test the project systematically and correct defects.

Instructions to the learner

- ☐ Test each functional requirement.
- ☐ Test invalid, empty and boundary inputs.
- ☐ Record expected versus actual results.
- ☐ Use Developer Tools to investigate failures.
- ☐ Correct defects and retest.
- ☐ Record at least one improvement made after testing.

Required evidence

- ☐ Completed test report.
- ☐ Debugging screenshots.
- ☐ Defect/fix record.
- ☐ Updated code and commits.

Competency criteria

- ☐ All main requirements are tested.
- ☐ Defects are described and corrected.
- ☐ Retesting confirms fixes.
- ☐ Evidence supports the final quality of the project.

Learner Activity Record

Date started	
Date completed	
Location / platform	
Time spent (hours)	
GitHub repository / link	
Evidence file names	
Challenges experienced	
How challenges were resolved	

Learner Reflection

What did you complete in this activity?

Which JavaScript concepts did you apply?

What worked well?

What error or difficulty did you encounter?

How would this skill be useful in a retail or workplace environment?

Facilitator / Assessor Verification

Criterion	Competent	Not Yet Competent	Comments / corrective action
All main requirements are tested.	☐	☐	
Defects are described and corrected.	☐	☐	
Retesting confirms fixes.	☐	☐	
Evidence supports the final quality of the project.	☐	☐	

Overall result: Competent / Not Yet Competent	
Assessor / facilitator name	
Signature	
Date	
Learner acknowledgement and signature	

Activity 4.4: Final Demonstration, Presentation and Reflection

Purpose of the Activity

Present the completed project and explain how JavaScript was used.

Instructions to the learner

- ☐ Demonstrate the main user flow.
- ☐ Explain HTML, CSS and JavaScript responsibilities.
- ☐ Explain at least three JavaScript concepts used.
- ☐ Show GitHub evidence.
- ☐ Discuss one challenge, one limitation and one future improvement.
- ☐ Submit the final reflection.

Required evidence

- ☐ Presentation or demonstration evidence.
- ☐ Final screenshots.
- ☐ Final reflection.
- ☐ Facilitator/assessor observation record.
- ☐ Final repository link.

Competency criteria

- ☐ Project runs and is demonstrated confidently.
- ☐ Learner explains the solution accurately.
- ☐ Evidence is complete and organised.
- ☐ Reflection identifies meaningful learning and improvement.

Learner Activity Record

Date started	
Date completed	
Location / platform	
Time spent (hours)	
GitHub repository / link	
Evidence file names	
Challenges experienced	
How challenges were resolved	

Learner Reflection

What did you complete in this activity?

Which JavaScript concepts did you apply?

What worked well?

What error or difficulty did you encounter?

How would this skill be useful in a retail or workplace environment?

Facilitator / Assessor Verification

Criterion	Competent	Not Yet Competent	Comments / corrective action
Project runs and is demonstrated confidently.	☐	☐	
Learner explains the solution accurately.	☐	☐	
Evidence is complete and organised.	☐	☐	
Reflection identifies meaningful learning and improvement.	☐	☐	

Overall result: Competent / Not Yet Competent	
Assessor / facilitator name	
Signature	
Date	
Learner acknowledgement and signature	

Use one row for each facilitated practical session, workshop, supported lab or approved workplace practice session.

[illegible]

Date	Module / activity	Start	End	Hours	Summary of work completed	Learner initials	Facilitator initials

Corrective Action and Reassessment Register

[illegible]

Final Portfolio of Evidence Checklist

Module	Required evidence	Submitted	Authentic	Accepted	Assessor comments
PM-01	Project folder structure and three source files	☐	☐	☐	
PM-01	Working first interactive page screenshots	☐	☐	☐	
PM-01	Developer Tools / console evidence	☐	☐	☐	
PM-01	GitHub repository, commits and branch/PR evidence	☐	☐	☐	
PM-02	Variables, types and operators exercises	☐	☐	☐	
PM-02	Conditions and functions exercises	☐	☐	☐	
PM-02	Arrays and loops exercises	☐	☐	☐	
PM-02	Support Ticket Priority Checker and screenshots	☐	☐	☐	
PM-03	Debugging and error-handling evidence	☐	☐	☐	
PM-03	Department Contact Form with validation	☐	☐	☐	
PM-03	DOM and events evidence	☐	☐	☐	
PM-03	API data display and error handling	☐	☐	☐	
PM-04	Approved project plan and wireframe	☐	☐	☐	
PM-04	Final project source files and repository	☐	☐	☐	
PM-04	Test and debugging report	☐	☐	☐	
PM-04	Presentation evidence and final	☐	☐	☐	

Module	Required evidence	Submitted	Authentic	Accepted	Assessor comments
	reflection				

These concepts introduce dynamic decision-making, reusable program functions,

Final Module Results and Sign-off

Module	Credits	Result	Assessment date	Assessor signature	Moderator / reviewer
PM-01	6	Competent / Not Yet Competent			
PM-02	6	Competent / Not Yet Competent			
PM-03	12	Competent / Not Yet Competent			
PM-04	12	Competent / Not Yet Competent			

Integrated Practical Completion Declaration

The undersigned confirm that the learner's practical evidence has been reviewed against the applicable practical module requirements and provider assessment instruments. Any outstanding gaps, reassessments or conditions are recorded in this logbook.

Learner name and signature	
Facilitator / mentor name and signature	
Registered assessor name and signature	
Moderator / quality reviewer name and signature	
Employer / workplace representative (where applicable)	
Final completion date	

Reference and Governance Note

This logbook was developed with reference to QCTO guidance that occupational and skills programmes are workplace- and practice-oriented, that providers assess and record results for knowledge and practical modules, and that assessment evidence must support judgements of learner achievement. The exact implementation must remain consistent with the registered curriculum, provider accreditation scope, assessment policy, applicable Quality Partner/SETA requirements and any approved learning-material matrix.

- Quality Council for Trades and Occupations (QCTO). Skills Programmes and provider guidance.
- QCTO. The Road to Success: Skills Development Provider Guide.

- QCTO. Policy on Accreditation of Skills Development Providers.
- QCTO. Assessment Policy and Qualification Assessment Specification guidance.
- redAcademy JavaScript Fundamentals Curriculum and PM-01 to PM-04 learning content.